Interspeech 2025
17-21 August 2025, Rotterdam, The Netherlands

Adapting Whisper for low-resource Hindi-English Code-Mix speech with on-the-fly Augmentation & LLM-Synthesised Data

Astik Biswas, Oleg Shevelev, Amine Abdaoui, Vivek Tyagi, Abdelmoumene Boumadane

OCI Speech, Oracle Cloud Infrastructure

astik.biswas@oracle.com, oleg.shevelev@oracle.com, amin.abdaoui@oracle.com,
vivek.v.tyagi@oracle.com, abdelmoumene.boumadane@oracle.com

Abstract

Code-switching (CS) automatic speech recognition (ASR) faces challenges due to language confusion from accents, acoustic similarities, and seamless transitions. In multilingual India, CS is prevalent, yet adapting pre-trained Whisper for low-resource Indian CS-ASR remains under-explored. This study explores strategies for adapting Whisper with limited data. First, we propose language prompts for fine-tuning and on-the-fly code-mixed data simulation to handle language switches. Second, we use Llama for few-shot code-switching (CS) text generation, coupled with audio synthesis, to create synthetic data for fine-tuning the Whisper model. Experiments on a Hindi-English CS dataset show promising results, demonstrating the techniques effectiveness. These findings are applicable to other multilingual contexts, aiding Whisper's adaptation to new domains.

Index Terms: code-mix speech recognition, whisper, Llama, augmentation

1. Introduction

Code-switching, the alternation between two or more languages within a discourse, is a common phenomenon in multilingual societies. In India, with 22 official languages and English as the *lingua franca*, code-mixing often occurs through alternation and insertion [1]. Speakers frequently switch between English, a high-resource language, and their native, under-resourced languages. This practice is prevalent across diverse domains such as banking, real estate, healthcare, and education, highlighting the unavoidable role of English as a bridge language usually.

The prevalence of code-switching presents a significant challenge for Automatic Speech Recognition (ASR) systems, as it demands nuanced multilingual understanding [1]. Current ASR models, including state-of-the-art systems like Whisper, struggle with code-switching speech due to limited labeled data [2], particularly in Indic languages, where seamless language transitions and data scarcity exacerbate the difficulty for transformer-based architectures.

Recent research has addressed challenges in code-switching automatic speech recognition (CS-ASR), particularly data scarcity, through three technical domains: speech [1, 3, 4], text [5, 6], and modeling [7, 8]. Speech approaches integrate monolingual data [1] into CS-ASR systems and enhance pronunciation models [9] to handle accents and variations. Text methods augment code-switching text from monolingual corpora [1] or generating synthetic text [10] to improve language models. Modeling innovations include Mixture of Experts (MoE) [7] frameworks with language-specific encoders and decoders, and auxiliary tasks like frame-level language identification [8].

The whisper large-v2 model [11] is a state-of-the-art multilingual ASR system excelling on mono-lingual speech samples but struggles with code-mixed ones, often leading to deletions when a switch to a different language occurs. Recent efforts to improve its code-mixing capabilities include Zhao et al. [12], who introduced an encoder refiner with Long Short-Term Memory (LSTM)/Connectionist Temporal Classification (CTC) architectures and language-aware adaptation, reducing MER by 6.3% on the SEAME dataset [13]. However, latency overhead remains unaddressed. PromptingWhisper [14] achieved a 19% relative gain using zero-shot prompting, while Yang et al. [15] enhanced performance by integrating fused language embeddings and fine-tuning with code-mixed data, effectively addressing Mandarin-English code-switching challenges.

Users have reported challenges in processing code-mixed speech with Whisper models, which remains an open issue[1]. While speaker diarization and audio segmentation have been suggested, these methods introduce latency and are impractical for low-latency applications. Moreover, they fail when a single speaker switches languages within the same segment. This issue is particularly critical for Indic languages, where highly dynamic and spontaneous code-mixing demands further research and refinement. Most existing work on Whisper adaptation for code-mixed ASR has focused on the English-Mandarin use case, with no known studies on Hindi-English adaptation. This paper explores strategies to leverage Whisper for the under-resourced Hindi-English CS-ASR task and examines whether its fine-tuning behavior aligns with Mandarin-English. We propose methods to address the scarcity of code-mix training data across domains and and seek answers to the following questions, structured into two study tracks.

1. Track 1: Fine-tuning with limited in-domain CS Data
   - Can a small fraction of in-domain CS data (considering limited training data availability), combined with monolingual data, improve CS-ASR performance?
   - Does on-the-fly code-mix simulation enhance the model's ability to capture language switch points?
   - Does using language prefixes during training improve model performance?
2. Track 2: Fine-tuning with Synthetic Data via LLM and TTS
   - Can Llama [16] from the meta large language models (LLMs) family effectively generate domain-specific code-mixed utterances through few-shot prompting to adequately cover a wide range of CS points matching target domain, with synthetic code-mixed audio produced using text-to-speech (TTS) systems?

[1] https://github.com/openai/whisper/discussions/49

10.21437/Interspeech.2025-447

Table 1: *Statistics of Hindi-English code-mix data. $Words_H$ and $Words_E$ represents Hindi and English words respectively; HE represents Hindi-English; EH represents English-Hindi*

Set	Duration (hours)	$Words_H$ (No.)	$Words_E$ (No.)	Total Words	No. of HE CS	No. of EH CS	Total CS
$Train$	89.8	445,762	160,694	606,456	85761	90802	176,563
$Train_{sub}$	15	89,654	32,073	121,727	17032	18049	35,081
$Test$	5.2	28,215	9,627	37,842	4189	5176	9,365

- Can monolingual and synthetic code-mix audio, in place of real training data, effectively adapt the model to the target domain?

Although TTS-generated synthetic data is commonly used to enhance ASR quality [17], the novelty of Track 2 lies in generating in-domain synthetic code-mixed data via LLM prompting.

2. Data

This study utilizes the Hindi-English code-mixed dataset from the Multilingual and Code-Switching ASR Challenge [18], sourced from spoken tutorials on technical topics, where code-switching arises predominantly from the lecture content. Table 1 outlines the complete training set ($Train$), a smaller randomly selected subset ($Train_{sub}$) for Track 1 experiments, and the test set ($Test$). While no ablation study has been conducted to determine the sufficiency of in-domain code-mixed data [15], this research aims to address data scarcity and evaluate the effectiveness of proposed training strategies.

The dataset allows to grasp the frequency of code-mixing in spontaneous speech for an Indic language, underscoring the challenges faced by ASR systems in seamlessly handling language transitions at code-switching (CS) points. Specifically, the test set includes 4,189 Hindi-English and 5,176 English-Hindi CS bigrams, with unique counts of 2,347 and 2,472, respectively.

In addition to the in-domain data, we incorporated 15 hours each of Hindi ($Train_H$) and English ($Train_E$) monolingual out-of-domain (OOD) data from the Common Voice corpus [19]. The primary objective of using this OOD data was to evaluate whether transformer-based ASR models, such as Whisper, could benefit from such data, as suggested by prior research on the legacy hybrid ASR [20]. A secondary objective is to leverage this monolingual corpus to simulate on-the-fly code-mixing, as proposed in Track 1.

3. Methodology

As outlined in the introduction, we seek a solution that assumes minimal or no target training data. In Track 1, we explore fine-tuning techniques and on-the-fly code-mix simulation using monolingual data, with or without a small subset of training data ($Train_{sub}$). Track 2 employs LLaMA2 few-shot prompting to generate target-domain-like synthetic text, followed by TTS-based voice synthesis. Both pipelines are illustrated in Fig.1.

3.1. Track 1: On-the-fly code-mix simulation

This work proposes a simple and efficient technique to simulate code-mixed data from monolingual speech during training. Since off-the-shelf models often perform poorly due to the lack of code-mixed data, this approach addresses the issue by generating synthetic code-mixed samples. Collecting real code-switch audio with transcriptions is time-consuming and costly; hence, this method leverages existing monolingual audio for on-the-fly data augmentation during training as described in algorithm 1. By randomly concatenating audio and transcriptions with different language tags within each batch, the model was exposed to diverse code-switch points, improving its robustness to language changes. Metadata, including language tags, guiding this process, ensured that new audio samples meet duration and token length constraints for seamless Whisper fine-tuning.

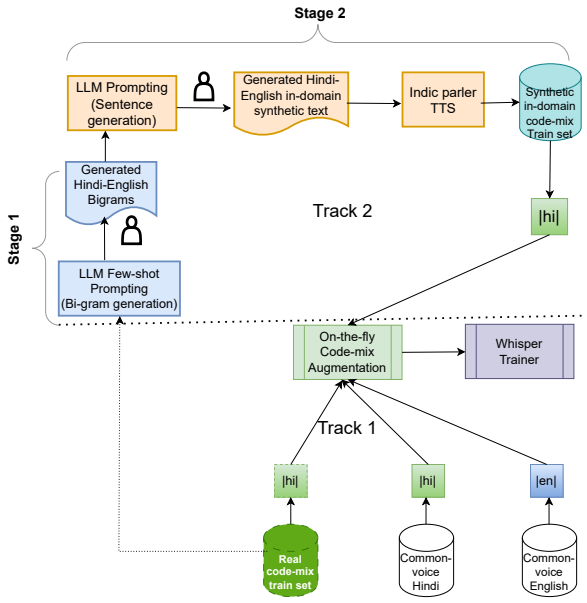

Figure 1: *Proposed pipeline for Whisper finetuning targeting code-mix speech. Track 1: On-the-fly code-mixing with language specific prompting; Track 2: Synthetic in-domain code-mix data generation leveraging few shot prompting of LLM*

Algorithm 1 Proposed On-the-fly CS simulation for Whisper

for $batch = 1, 2, \ldots$ **do**
  for $wav1 = 1, 2, \ldots$ **do**
    if $wav1[audio] < 30s$ **then**
      if $wav1[lang\_tag] = en$ **then**
        $target\_lang\_tag = hi$
      else
        $target\_lang\_tag = en$
      end if
      $wav2 = select\ randomly\ with$ **target_lang_tag**
      $wav3 = wav1$ **concat** $wav2$
      $wav3[lang\_tag] = wav1[lang\_tag]$
      if $wav3[audio] > 30s\ OR\ wav3[label] > 448\ Chars$ **then**
        $Revert\ to\ original,\ wav3 = wav1$
      end if
      $Pass$ **wav3** $to\ trainer$
    end if
  end for
end for

3.2. Track 2: Leveraging LLM to generate in-domain synthetic CS data

This track is implemented in two stages: (1) leveraging LLMs to generate domain-aligned code-mixed text and (2) using TTS to create in-domain synthetic audio, enhancing ASR's code-switching capabilities. We employ Llama-3.3-70B-Instruct, a

state-of-the-art open-source LLM.

3.2.1. Stage 1: Bigram generation & Filtering

As a first stage, we generated code-mix bigrams leveraging prompt engineering using Llama as shown below:

> *Role (System)*: "You're a helpful mix-bigrams generator, fluent in both English and Hindi, and that only prints out the bigrams and no additional comments."
>
> *Role (User)*: "Generate 10 **language mix bigrams** in Hindi-English. Each bigram is a couple of different words that usually go together and often written this way: one in Hindi and the other one in English. **Never** put just a translation of one word next to each other. Hindi words should be in **Devnagari** and English words should be in **Latin**. Follow these examples: प्रस्तुति document; बुनियादी formatting; इस spoken. Use these phrases with mixed language for inspiration. **Use different words for bigrams different from the ones in the text**: $< 5\ sentences\ a\ mix\ of\ English\ and\ Hindi\ in\ them >$"

The Llama was repeatedly prompted with batches of five Hindi-English sentences from the training set using a few-shot approach to generate domain-specific synthetic code-mix bigrams. Generating bigrams instead of full sentences ensured greater control over the process, allowing for easier quality checks and the filtering of suboptimal outputs.

Using several thousand in-domain sentences, we generated 44,657 code-mixed bigrams. After removing duplicates, only 5,932 unique bigrams remained (13.3% of the Llama-generated output). Manual evaluation revealed that some bigrams consisted entirely of English or Hindi words, or included an English word followed by its Hindi translation (or vice versa). To address this, we implemented a script that filters out bad bigrams to ensure each bigram contained both Hindi and English characters and used the Llama to verify that paired words were not direct translations. This refinement resulted in 5,477 valid bigrams which is 93.3% of the unique set.

3.2.2. Stage 2: Code-mix sentence and audio generation

We again leveraged Llama with following prompt to generate four sentences for each good bigram generated in last stage:

> *Role (System)*: "You're a helpful sentence generator, fluent in both English and Hindi, and that output only required sentences without any additional comments."
>
> *Role (User)*: "Generate four sentences that would use this bigram in Hindi-English in a natural way. Make 2 sentences with English as the main language and 2 sentences with Hindi as the main language. Word bigram to use: $< bigram >$"

This process generated approximately 16,000 unique synthetic sentences. While generating four sentences per bigram would theoretically yield 21,908 sentences (5,477 × 4), manual review identified instances where the LLM deviated from the requested number of sentences, reducing the total for certain bigrams. Nevertheless, the dataset should be adequate for evaluating the effectiveness of this approach in enhancing performance through fine-tuning. Parler-TTS[2] was used to generate code-mixed audio with one Indian male and one Indian female voice, creating two synthetic training sets: $Train_{T1}$ (8 hours) and $Train_{T2}$ (22 hours), where $Train_{T1}$ is a subset of $Train_{T2}$.

Note to readers: This pipeline employs in-domain training examples as few-shot prompts for the LLM, though handcrafted text or domain-specific web-crawled data can serve as alternatives. The proposed framework is adaptable, enabling fully automated synthetic data generation for specific use cases.

4. Experiments

All experiments utilized the Whisper-large-v2 multilingual model, a Transformer-based encoder-decoder with 1.54 billion parameters, employing a sequence-to-sequence approach on log-Mel spectrograms. The model processes up to 30 second audio segments, with chunking for longer inputs. Training was conducted on two H100 GPUs (80GB VRAM each) with a batch size of 64 and gradient accumulation of 1. Optimization used AdamW with a peak learning rate of 2e-5, a maximum of 5k update steps, and mixed-precision training.

A four-hour subset is randomly reserved from the train set as a development set to monitor training performance on each checkpoint. Models were trained sequentially, with performance evaluations guiding subsequent training to achieve improved results. Ultimately, we present eleven models[3], as summarized in Table 2. To establish a baseline, we initially fine-tuned two Whisper large-v2 models (LB1 and LB2) using the complete in-domain training dataset (Table 1) with language tokens $< |en| >$ and $< |hi| >$, respectively, following the methodology in [15].

For Track 1, we report four models:

- M1 and M2: Trained with $< |hi| >$ as the language prompt, without and with on-the-fly code-mix simulation, respectively, using the monolingual data pool.
- M3: Trained with the monolingual data pool augmented by a small subset of in-domain training data ($Train_{sub}$). Trained with $< |hi| >$ as the language prompt with on-the-fly code-mix activated.
- M4: Trained with on-the-fly code-mix activated using language-specific prompts: $< |en| >$ for English monolingual and synthetic English-Hindi code-mixed speech, $< |hi| >$ for Hindi monolingual and synthetic English-Hindi code-mixed speech, while all real code-mixed speech ($Train_{sub}$) used $< |hi| >$.
- M5: The fine-tuning strategy mirrored that of M4, with the exception that the entire in-domain CS dataset was utilized rather than a limited subset. This approach represents an ideal scenario, leveraging the complete available real training data.

In Track 2, the real in-domain training set was left unaltered, and three models were trained: M6 with $Train_{T1}$, M7 with $Train_{T2}$, and M8 adding monolingual speech dataset. M8 was specifically trained to investigate whether OOD monolingual data could enhance a model previously fine-tuned with synthetic in-domain audio. Building on insights from Tracks 1 and 2, we fine-tuned a combined model (M9) incorporating on-the-fly code-switching to leverage the advantages of both approaches.

[2] https://huggingface.co/ai4bharat/indic-parler-tts

[3] Additional models with varying language prompts were tested, but only relevant results are reported.

Table 2: *Summary of model configurations fine-tuned with MER (%, lower better) and CBA (%, higher better) on $Test$ set.*

Model	Training data		Description	MER(%)↓	CBA(%)↑	
	Mono-lingual	code-mix			HE	EH
Large-V2	NA	NA	SOTA pre-trained Whisper model	52.0	42.9	36.8
LB1 (Baseline)	NA	$Train$	Lower bound using Language prompt <en>	44.7	57.4	56.0
LB2 (Baseline)	NA		Lower bound using Language prompt <hi>	37.4	64.1	66.4
M1 (Track 1)	$Train_H$,	NA	Mono-lingual data using Language prompt <hi>	56.8	29.3	29.7
M2 (Track 1)	$Train_E$		Mono-lingual data using Language prompt <hi>; on-the-fly CS activated	54.6	32.0	32.9
M3 (Track 1)	$Train_H$,	$Train_{Sub}$	Mono-lingual data using Language prompt <hi>; on-the-fly CS activated	45.6	53.6	53.7
M4 (Track 1)	$Train_E$		Mono-lingual data using Language specific prompt; on-the-fly CS activated 'en' for English and English-Hindi, 'hi' for Hindi, and Hindi-English	39.0	58.9	57.7
M5 (Track 1)	$Train_H$, $Train_E$	$Train$	Same as M4, but utilized all in-domain train set	**28.8**	**76.1**	**75.3**
M6 (Track 2)	NA	$Train_{T1}$	In-domain CS synthetic data, Language prompt <hi>	48.2	45.5	45.3
M7 (Track 2)		$Train_{T2}$		40.8	52.3	55.4
M8 (Track 2)	$Train_H$,	$Train_{T2}$	In-domain CS synthetic data and mono-lingual data, Language prompt <hi>	39.2	55.4	56.5
M9 (Track 1+2)	$Train_E$		In-domain CS synthetic data and mono-lingual data using language specific prompt; on-the-fly CS activated	35.9	60.0	60.2

5. Result & Discussions

Model performance was evaluated on the $Test$ set, with Mixed Language Word Error Rate (MER) [21] reported in Table 2. Decoding utilized WhisperX [22] with the "None" option for language detection during inference. To assess Hindi-English code-switched ASR performance, we analyze system accuracy at code-switch points using Code-Switch Bigram Accuracy (CBA), which measures correctly recognized bigrams at switch points relative to their total count in the test set. Table 2 presents CBA scores to compare model performance on code-switching.

The results indicate that while the pretrained large-v2 model is state-of-the-art (SOTA) multi-lingual model, it struggles with seamless Hindi-English code-mixing and MER is on higher side. The lower-bound baselines (LB1 and LB2), trained on the in-domain dataset, achieved 14% and 28.1% error reductions, respectively, over the pretrained model. These serve as lower-bound references, as the full $Train$ set was used to fine-tune Whisper. Furthermore, the CBA for both Hindi-English (HE) and English-Hindi (EH) scenarios showed substantial improvement. As demonstrated by [15] on Mandarin-English CS speech, while prompting strategies yield varied performances before adaptation, finetuning the Whisper model with code-switching speech data leads to uniformly enhanced performance, simplifying code-switching complexities. Interestingly, in our Hindi-English use case, the $< |hi| >$ language prompt outperformed $< |en| >$, highlighting the critical role of language prompts in Whisper fine-tuning. This suggests that language prompts must be carefully considered, as their effectiveness may vary across use cases.

As expected, M1, trained on OOD monolingual Hindi and English data, shows no improvement and exhibits an 9.2% relative regression compared to the pretrained model. With on-the-fly code-mixing activated, M2 demonstrated a 3.7% relative improvement over M1, supported by a 9% and 10.8% relative gain in CBA across EH and HE CS points, respectively. Models M3 and M4, fine-tuned with a subset of in-domain data ($Train_{sub}$) and on-the-fly code-mixing, show significant gains, with M4 achieving a 25% relative improvement over the pretrained model and notable CBA enhancements using language-specific prompts. While M4 does not surpass the baseline LB2, its results are comparable, demonstrating the efficacy of combining limited in-domain data (one-sixth of $Train$) with OOD monolingual data. M5, trained on the full in-domain dataset (considered an ideal condition) using the same strategy as M4, significantly outperforms baseline LB2 by 23% relative, establishing the efficacy of this approach. As expected, M5 achieves the best performance in the table 2, having been fine-tuned on the entire in-domain training set.

Focusing on Track 2 models (M6, M7), we observe improvements in MER and CBA with synthetic data. Model M6, trained on 8 hours of synthetic data ($Train_{T1}$), achieves a 7.3% relative MER improvement over the pretrained, and adding 14 more hours (M7) yields a further 15.3% relative improvement. Model M8, utilizing out-of-domain (OOD) monolingual and synthetic data, achieves a 3.9% relative improvement over M7. Enhancements in the CBA metric at code-switching points highlight the benefits of OOD monolingual data. Finally, M9, combining Track 1 and Track 2 strategies, shows a significant 8.4% relative improvement over M8. These results demonstrate that even without in-domain original training data, synthetic and OOD monolingual data enhance robustness, particularly when language-specific prompts and on-the-fly code-mixing are employed. The latter likely enables better learning of code-switch points during fine-tuning, improving performance under seamless code-mixinging. Both, track 1 and 2 demonstrates that on-the-fly code-mixing is most effective when supplemented with a some amount of in-domain real or synthetic data.

6. Conclusion & Future Work

In this paper, we proposed two techniques to handle seamless code-mixing, particularly in the Indian multilingual context. Our experiments demonstrate that both pipelines effectively outperform the baseline, achieving a 31% relative improvement over the pretrained model without using real in-domain data for Whisper fine-tuning. Leveraging language-specific prompting with on-the-fly code-mixing and few-shot prompting of LLMs for in-domain synthetic data generation significantly enhances performance in low-resource code-switching scenarios. While tested on Whisper large-v2 for Hindi-English tutorial speech, the approach is adaptable to other speech toolkits, domains, and other Whisper model variants. Both proposed approaches are straightforward, adaptable, and easily customizable. Additionally, the synthetic data generation pipeline and prompts can be further refined to enhance synthetic data quality. Future work could explore ablation studies to quantify the impact of synthetic data and refine on-the-fly code-mixing techniques by incorporating more controlled conditions, such as background similarity, speaker alignment if possible, and multiple synthetic switches per training example.

7. Acknowledgment

We would like to express our sincere gratitude to the leadership team—Sujith, Dan, Serge, and Nathan—for their invaluable guidance, support, feedback and encouragement throughout this research. We also extend our appreciation to Oracle for providing the resources and infrastructure that made this study possible.

8. References

[1] A. Biswas, E. Yılmaz, E. van der Westhuizen, F. de Wet, and T. Niesler, "Code-switched automatic speech recognition in five south african languages," *Computer Speech & Language*, vol. 71, p. 101262, 2022.

[2] A. Kulkarni, A. Kulkarni, M. Couceiro, and H. Aldarmaki, "Adapting the adapters for code-switching in multilingual ASR," *arXiv preprint arXiv:2310.07423*, 2023.

[3] X. Yue, G. Lee, E. Yılmaz, F. Deng, and H. Li, "End-to-end code-switching asr for low-resourced language pairs," in *Proceedings of ASRU*. IEEE, 2019, pp. 972–979.

[4] T. Verma, A. Shree, and A. Modi, "Asr for low resource and multilingual noisy code-mixed speech," *Proceedings of Interspeech*, 2023.

[5] Y. Li and P. Fung, "Code-switch language model with inversion constraints for mixed language speech recognition," in *Proceedings of COLING*, 2012, pp. 1671–1680.

[6] J. J. van Vüren and T. Niesler, "Improving under-resourced code-switched speech recognition: Large pre-trained models or architectural interventions," *Proceedings of the Interspeech, Dublin, Ireland*, pp. 20–24, 2023.

[7] Y. Lu, M. Huang, H. Li, J. Guo, and Y. Qian, "Bi-encoder transformer network for mandarin-english code-switching speech recognition using mixture of experts." in *Proceedings of Interspeech*, 2020, pp. 4766–4770.

[8] H. Liu, H. Xu, L. P. Garcia, A. W. Khong, Y. He, and S. Khudanpur, "Reducing language confusion for code-switching speech recognition with token-level language diarization," in *Proceedings of ICASSP*. IEEE, 2023, pp. 1–5.

[9] Y. Long, S. Wei, J. Lian, and Y. Li, "Pronunciation augmentation for mandarin-english code-switching speech recognition," *EURASIP Journal on Audio, Speech, and Music Processing*, vol. 2021, no. 1, p. 34, 2021.

[10] Y. Gao, J. Feng, Y. Liu, L. Hou, X. Pan, and Y. Ma, "Code-switching sentence generation by bert and generative adversarial networks." in *Proceedings of Interspeech*, 2019, pp. 3525–3529.

[11] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proceedings of ICML*. PMLR, 2023, pp. 28 492–28 518.

[12] J. Zhao, H. Shi, C. Cui, T. Wang, H. Liu, Z. Ni, L. Ye, and L. Wang, "Adapting Whisper for Code-Switching through Encoding Refining and Language-Aware Decoding," *arXiv preprint arXiv:2412.16507*, 2024.

[13] D.-C. Lyu, T. P. Tan, E. Chng, and H. Li, "SEAME: a Mandarin-English code-switching speech corpus in south-east asia." in *Proceedings of Interspeech*, vol. 10, 2010, pp. 1986–1989.

[14] P. Peng, B. Yan, S. Watanabe, and D. Harwath, "Prompting the Hidden Talent of Web-Scale Speech Models for Zero-Shot Task Generalization," in *Proceedings of Interspeech*, 2023.

[15] Y. Yang, Y. Peng, X. Zhong, H. Huang, and E. S. Chng, "Adapting OpenAI's Whisper for Speech Recognition on Code-Switch Mandarin-English SEAME and ASRU2019 Datasets," *arXiv preprint arXiv:2311.17382*, 2023.

[16] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[17] C.-T. Do, S. Imai, R. Doddipatla, and T. Hain, "Improving accented speech recognition using data augmentation based on unsupervised text-to-speech synthesis," in *Proceedings of EUSIPCO*. IEEE, 2024, pp. 136–140.

[18] A. Diwan, R. Vaideeswaran, S. Shah, A. Singh, S. Raghavan, S. Khare, V. Unni, S. Vyas, A. Rajpuria, C. Yarra *et al.*, "Multilingual and code-switching ASR challenges for low resource indian languages," in *Proceedings of Interspeech*, 2021, p. 2446–2450.

[19] R. Ardila, M. Branson, K. Davis, M. Henretty, M. Kohler, J. Meyer, R. Morais, L. Saunders, F. M. Tyers, and G. Weber, "Common voice: A massively-multilingual speech corpus," *arXiv preprint arXiv:1912.06670*, 2019.

[20] A. Biswas, E. van der Westhuizen, T. Niesler, and F. de Wet, "Improving asr for code-switched speech in under-resourced languages using out-of-domain data," in *Proceedings of SLTU*, 2018, pp. 122–126.

[21] S. Zhang, J. Yi, Z. Tian, J. Tao, Y. T. Yeung, and L. Deng, "Reducing multilingual context confusion for end-to-end code-switching automatic speech recognition," in *Proceedings of Interspeech*, 2022, pp. 3894–3898.

[22] M. Bain, J. Huh, T. Han, and A. Zisserman, "WhisperX: Time-accurate speech transcription of long-form audio," in *Proceedings of Interspeech*, 2023, pp. 4489–4493.